CLASE 3 · PROGRAMACIÓN EN BASH

```
usuario@clase:~$ if · read · while
```

Decidir, preguntar, repetir

Condicionales, scripts interactivos y bucles

Basado en los capítulos 7 a 9 de *Bash Guide for Beginners*, de Machtelt Garrels

AGENDA

Del guion fijo al programa

07

Condicionales

`if · test · case`

Ejecutar una cosa u otra según el resultado de un comando o una comparación.

08

Scripts interactivos

`echo · read · exec`

Mostrar mensajes, pedir datos y redirigir entrada y salida.

09

Tareas repetitivas

`for · while · select`

Bucles, menús y argumentos en cantidad variable.

CAPÍTULO 7

Condicionales

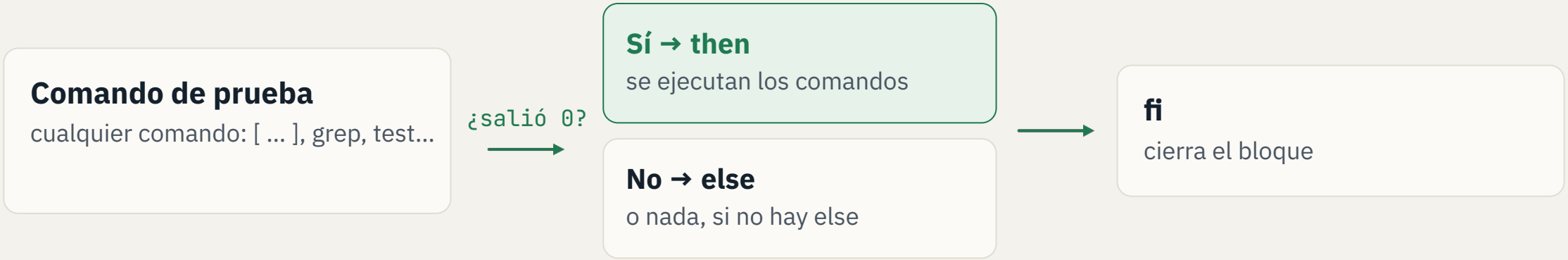
if, test y case: elegir qué hacer según el resultado de un comando.

07

7.1 · INTRODUCCIÓN A IF

if ejecuta según un código de salida

```
if COMANDOS-DE-PRUEBA; then COMANDOS; fi
```



La condición no tiene que ser una comparación: **cualquier comando** que devuelva un código de salida sirve.

7.1.1.1 · EXPRESIONES PRIMARIAS

Probar archivos con [ ]

`-e ARCH`	Existe	`-f ARCH`	Existe y es un archivo regular
`-d ARCH`	Existe y es un directorio	`-L ARCH`	Es un enlace simbólico
`-r ARCH`	Se puede leer	`-w ARCH`	Se puede escribir
`-x ARCH`	Es ejecutable	`-s ARCH`	Tamaño mayor que cero
`A -nt B`	A es más nuevo que B		

```
if [ -f /var/log/messages ]
then
  echo "El archivo existe."
fi
```

Los espacios dentro de los corchetes son obligatorios: **[** es un comando y cada parte es un argumento.

7.1.1.1 · COMPARACIONES

Comparar cadenas y números

Cadenas

`-z CAD`	Longitud cero (vacía)
`-n CAD`	Longitud distinta de cero
`A == B`	Iguales (= en POSIX)
`A != B`	Distintas
`A < B`	A va antes en orden alfabético

```
[ "$(whoami)" != "root" ]
```

Números enteros

`-eq`	Igual	`-ne`	Distinto
`-lt`	Menor	`-le`	Menor o igual
`-gt`	Mayor	`-ge`	Mayor o igual

```
[ "$num" -gt 150 ]
```

7.1.2 · CÓDIGO DE SALIDA

Decidir según el éxito de un comando

Con $?

```
grep "$USER" /etc/passwd
if [ $? -ne 0 ]; then
  echo "No es una cuenta local"
fi
```

Directamente con el comando

```
if ! grep -q "$USER" /etc/passwd; then
  echo "No es una cuenta local"
fi
```

Forma corta: && y ||

```
[ "$(whoami)" ≠ "root" ] && {
  echo "Se necesita ser root"
  exit 1
}
```

&& ejecuta lo siguiente si hubo éxito; **||**, si hubo fallo.

Usar **{ }** y no **( )**: los paréntesis abren una subshell y el **exit** no terminaría el script.

7.1.2.3 · TRES FORMAS DE PROBAR

[ ], [[ ]] y (( ))

[…]

test clásico

POSIX, funciona en cualquier shell sh. Las variables van entre comillas.

```
[ -f "$ARCH" ]
```

[[…]]

test de Bash

No divide palabras ni expande nombres de archivo. Con == compara contra un patrón.

```
[[ $resp == s* ]]
```

((…))

aritmética

Evalúa una expresión entera. Las variables no necesitan $.

```
(( anio % 4 == 0 ))
```

7.2 · IF MÁS COMPLETO

else y elif

```
if PRUEBA; then
  COMANDOS
elif OTRA-PRUEBA; then
  OTROS-COMANDOS
else
  ALTERNATIVA
fi
```

```
#!/bin/bash
if [ $# -ne 1 ]; then
  echo "Uso: $0 archivo"
  exit 1
fi
if [ -f "$1" ]; then
  echo "Tamaño: $(ls -lh "$1" | awk '{ print $5 }')"
elif [ -d "$1" ]; then
  echo "$1 es un directorio"
else
  echo "$1 no existe"
fi
```

Primero se valida la cantidad de argumentos (**$#**) y se muestra un mensaje de uso; después se prueba el archivo.

7.2.3 · ANIDAR Y COMBINAR

if anidados o una sola condición

Anidados

```
anio=$(date +%Y)
if (( anio % 400 == 0 )); then
  echo "Bisiesto"
elif (( anio % 4 == 0 )); then
  if (( anio % 100 ≠ 0 )); then
    echo "Bisiesto"
  else
    echo "No bisiesto"
  fi
else
  echo "No bisiesto"
fi
```

Con && y ||

```
anio=$(date +%Y)
if (( anio % 400 == 0 )) ||
   (( anio % 4 == 0 && anio % 100 ≠ 0 ))
then
  echo "Bisiesto"
else
  echo "No bisiesto"
fi
```

Mismo resultado, mucho más legible.

7.2.5 · EXIT

exit: devolver un código al que llamó

verificar.sh

```
#!/bin/bash
[ $# -eq 1 ] || exit 2   # uso incorrecto
[ -e "$1" ] || exit 1  # no existe
exit 0                  # todo bien
```

El script que lo llama

```
./verificar.sh datos.txt
case $? in
  0) echo "Listo" ;;
  1) echo "El archivo no existe" ;;
  2) echo "Uso: verificar.sh archivo" ;;
esac
```

0 = éxito

Cualquier otro número indica un fallo; el programador elige qué significa cada uno.

Sin argumento

exit devuelve el valor actual de **$?**.

Termina todo el script

Se usa ante datos incorrectos o un comando que falló.

7.3 · CASE

case: elegir entre muchos patrones

```
case EXPRESIÓN in
  PATRÓN1) COMANDOS ;;
  PATRÓN2|PATRÓN3) COMANDOS ;;
  *) POR DEFECTO ;;
esac
```

`)`	Cierra la lista de patrones
`|`	Separa patrones alternativos
`;;`	Termina cada cláusula
`*)`	Cualquier otro valor

disktest.sh (adaptado)

```
case $espacio in
  [1-6]*)
    msj="Todo tranquilo." ;;
  [7-8]*)
    msj="Hay que ir limpiando: $espacio%." ;;
  9[1-8])
    msj="¡Urge un disco nuevo! $espacio%." ;;
  99)
    msj="¡Me ahogo! $espacio%." ;;
  *)
    msj="Valor inesperado." ;;
esac
```

case para procesar una opción

```
case "$1" in
  start)
    start ;;
  stop)
    stop ;;
  restart)
    stop
    start ;;
  status)
    status anacron ;;
  *)
    echo "Uso: $0 {start|stop|restart|status}"
    exit 1 ;;
esac
```

Así arrancan y detienen servicios los scripts de inicio clásicos:

`servicio start` `servicio stop`

start y stop son **funciones** definidas en el mismo script (capítulo 11).

El caso ***)** muestra cómo usarlo y termina con un código de error.

CAPÍTULO 8

Scripts interactivos

Informar al usuario, pedirle datos y controlar de dónde se lee y adónde se escribe.

08

8.1 · ¿INTERACTIVO O NO?

¿Interactivo o automático?

No interactivo

- Se comporta igual cada vez.
- Puede correr en segundo plano o programado (cron).

Interactivo

- Más flexible.
- El usuario lo adapta mientras corre.
- Puede informar su progreso.

8.1.2 · ECHO

echo: mensajes para el usuario

`-e`	Interpreta las secuencias con barra invertida
`-n`	No agrega el salto de línea final

```
$ echo -e "Uso:\t$0 menú animal\n"
Uso:    ./feed.sh menú animal

$ echo -n "Nombre: "; read nombre
Nombre:
```

Secuencias con -e

`\n`	Salto de línea
`\t`	Tabulación
`\a`	Alerta (campana)
`\b`	Retroceso
`\c`	Sin salto de línea final
`\\`	Barra invertida

8.2.1 · READ

read: leer lo que escribe el usuario

```
read [opciones] VAR1 VAR2 … VARN
```

```
$ read nombre resto
Ada Lovelace Byron
$ echo "$nombre"
Ada
$ echo "$resto"
Lovelace Byron
```

Una palabra por variable; la última recibe todo lo que sobra. Sin variables, la línea queda en **REPLY**.

`-p TEXTO`	Muestra un mensaje antes de leer
`-s`	No muestra lo que se escribe
`-n N`	Lee solo N caracteres
`-t SEG`	Se corta tras SEG segundos
`-r`	La \ no es carácter de escape
`-a ARRAY`	Guarda las palabras en un array
`-d DELIM`	Termina en DELIM, no en salto de línea

8.2.2 · PEDIR DATOS

Pedir datos con un buen mensaje

bisiesto.sh

```
#!/bin/bash
read -p "Año a verificar (4 dígitos): " anio
if (( anio % 400 == 0 )) ||
   (( anio % 4 == 0 && anio % 100 ≠ 0 )); then
  echo "$anio es bisiesto."
else
  echo "$anio no es bisiesto."
fi
```

```
$ ./bisiesto.sh
Año a verificar (4 dígitos): 2000
2000 es bisiesto.
```

Otros usos

Una sola tecla:

```
read -n 1 -p "¿Seguir? [s/n] " r
```

Sin mostrar lo escrito:

```
read -s -p "Clave: " clave
```

Con límite de tiempo:

```
read -t 10 -p "Nombre: " n
```

8.2.3 · DESCRIPTORES DE ARCHIVO

Tres canales abiertos en cada proceso

0

stdin

Entrada estándar: por defecto, el teclado.

```
cmd < archivo
```

1

stdout

Salida estándar: por defecto, la pantalla.

```
cmd > archivo
```

2

stderr

Errores: también a la pantalla.

```
cmd 2> archivo
```

```
$ ls -l /dev/std*
/dev/stderr → ../proc/self/fd/2
/dev/stdin → ../proc/self/fd/0
/dev/stdout → ../proc/self/fd/1
```

8.2.3.2 · REDIRIGIR ERRORES

Redirigir errores: el orden importa

`cmd 2> errores.txt`	Solo los errores al archivo
`cmd > todo.txt 2>&1`	Salida y errores al mismo archivo
`cmd &> todo.txt`	Lo mismo, forma abreviada de Bash
`cmd 2> /dev/null`	Descartar los errores
`cmd 2>&1 > salida.txt`	¡Ojo! Solo stdout va al archivo

Se procesan de izquierda a derecha

2>&1 significa «el 2 apunta adonde apunta el 1 ahora». Si el 1 se redirige después, el 2 ya quedó en la pantalla.

CAPÍTULO 9

Tareas repetitivas

Bucles for, while y until, break y continue, menús con select y argumentos con shift.

09

9.1 · FOR

for: un comando por cada elemento

```
for VAR in LISTA; do COMANDOS; done
```

La lista puede venir de...

`a b c`	Palabras escritas a mano
`*.xml`	Comodines de archivos
`$(comando)`	La salida de un comando
`(sin in)`	Los argumentos, como "$@"

Copia de respaldo de cada .xml

```
for i in *.xml; do
  cp "$i" "$i.bak"
done
```

```
$ ls *.xml*
file1.xml  file1.xml.bak
file2.xml  file2.xml.bak
```

9.2 · WHILE

while: repetir mientras un comando tenga éxito

```
while COMANDO-DE-CONTROL; do COMANDOS; done
```

Contador

```
i=0
while [ $i -lt 4 ]; do
  echo "Vuelta $i"
  i=$((i + 1))
done
```

Para siempre, hasta Ctrl+C

```
while true; do
  touch "pic-$(date +%s).jpg"
  sleep 300
done
```

9.2.2.4 · UN EJEMPLO COMPLETO

Calcular un promedio

```
#!/bin/bash
SUMA=0; CANT=0
while true; do
  read -p "Puntaje [0-100] ('q' para salir): " P
  if [ "$P" == "q" ]; then
    break
  elif (( P < 0 || P > 100 )); then
    echo "Fuera de rango, intente de nuevo."
  else
    SUMA=$((SUMA + P)); CANT=$((CANT + 1))
  fi
done
(( CANT > 0 )) && echo "Promedio: $((SUMA / CANT))%"
```

break
Sale del bucle cuando el usuario escribe q.

Validar antes
Primero la q, después el rango: si no, «q» llega a la aritmética.

División entera
Bash solo hace aritmética entera.

9.3 · UNTIL

until: repetir hasta que un comando tenga éxito

```
until COMANDO-DE-PRUEBA; do COMANDOS; done
```

```
until [ -f /tmp/listo ]; do
  echo "Esperando..."
  sleep 5
done
echo "¡Listo!"
```

`while`

sigue mientras la prueba **tiene éxito**

`until`

sigue mientras la prueba **falla**

Misma sintaxis, condición invertida.

Alimentar un bucle con un archivo o una tubería

Desde un archivo: después de done

```
while IFS=: read -r usuario x uid resto; do
  (( uid ⩾ 1000 )) && echo "$usuario"
done < /etc/passwd
```

El bucle es un solo comando, así que la redirección va al final.

Desde una tubería

```
find . -type f -mtime +5 -print0 |
while IFS= read -r -d '' arch; do
  gzip "$arch"
  echo "$arch archivado"
done
```

Con **-print0** y **-d ''** funciona aunque los nombres tengan espacios.

break y continue

`break`

Sale del bucle actual (no del script).

`continue`

Salta a la siguiente vuelta.

Pasar nombres a minúsculas

```
for nombre in *; do
  [[ $nombre =≠ *[[:upper:]]* ]] && continue
  nuevo=$(echo "$nombre" | tr 'A-Z' 'a-z')
  if [ -e "$nuevo" ]; then
    echo "$nuevo ya existe, no se toca"
    continue
  fi
  mv "$nombre" "$nuevo"
done
```

9.6 · SELECT

select: menús en una línea

```
PS3="Elija una opción: "
select op in "Ver fecha" "Ver usuarios" "Salir"; do
  case $op in
    "Ver fecha")    date ;;
    "Ver usuarios") who ;;
    "Salir")        break ;;
    *) echo "Opción $REPLY no válida" ;;
  esac
done
```

```
1) Ver fecha
2) Ver usuarios
3) Salir
Elija una opción: 3
```

PS3	El mensaje que pide la opción
REPLY	Lo que escribió el usuario
break	La forma amable de salir

9.7 · SHIFT

shift: correr los argumentos a la izquierda

Antes

$1	$2	$3	$4
a	b	c	d

shift

Después: $1 se descarta

$1	$2	$3
b	c	d

Procesar cualquier cantidad de argumentos

```
if [ $# -eq 0 ]; then
  echo "Uso: $0 dir1 dir2 ..."; exit 1
fi
while (( $# )); do
  echo "Procesando $1"
  shift
done
```